

Exercício Programa 01 – Planilha Esparsa com Histórico de Alterações

Algoritmos e Estrutura de Dados I (2026)

Prof. Renan Cerqueira Afonso Alves

Prazo de entrega: 04/10/2026 às 23:59h

v1.1

1 Introdução

Planilhas eletrônicas, como o Microsoft Excel ou o Google Sheets, organizam dados em grades bidimensionais e estão presentes no dia a dia de praticamente qualquer área, da engenharia às ciências humanas. Na prática, no entanto, a grande maioria das células de uma planilha permanece vazia – ou, no contexto deste EP, com o valor 0. Armazenar todas as posições da grade em um arranjo bidimensional tradicional seria extremamente ineficiente em termos de memória, já que seria necessário reservar espaço para uma enorme quantidade de posições nulas, além de impor um limite fixo de linhas e colunas.

Uma alternativa muito mais eficiente é utilizar uma estrutura que armazene apenas as células cujo conteúdo é diferente de zero. Melhor ainda, essa estrutura pode ser projetada sem que seja necessário definir previamente um número máximo de linhas ou colunas: diferentemente de uma abordagem baseada em vetores de listas (um vetor para as linhas e outro para as colunas), a representação cresce dinamicamente conforme novas células são inseridas, tornando-a adequada para modelar planilhas de dimensões potencialmente ilimitadas e esparsas.

Além disso, todo editor de planilhas moderno oferece a operação de *undo* (desfazer). Uma pilha resolve esse problema de forma direta e elegante: a cada alteração, o estado anterior da célula afetada é empilhado e, ao desfazer, o topo da pilha é consumido e o estado anterior é restaurado.

Neste EP você irá implementar exatamente essa ideia: uma mini-planilha eletrônica representada por uma matriz esparsa em listas encadeadas cruzadas (linhas e colunas não vazias organizadas em listas ligadas, sem limite fixo de tamanho), combinada com uma pilha dinâmica que registra o histórico de alterações e permite desfazer a última operação realizada.

2 Objetivo

Você deve implementar, em C, com alocação dinâmica de memória (sem vetores de tamanho fixo para linhas, colunas ou histórico), uma biblioteca de planilha esparsa com as seguintes capacidades:

- definir e remover o valor de uma célula;
- consultar o valor de uma célula;
- somar os valores de um intervalo retangular de células;
- transpor um trecho da planilha;
- contar quantas células não nulas existem na planilha;
- desfazer a última alteração realizada (undo);
- exibir o estado atual da planilha; e
- exibir o estado atual do histórico de alterações.

2.1 Arquivos fornecidos

Foi fornecido no eDisciplinas um arquivo inicial (*template*) com a definição das structs explicadas na Seção 3 e com a assinatura das funções definidas na Seção 4. Não será permitido modificar as estruturas (adicionar, remover ou renomear campos) ou a assinatura das funções.

O template contém a implementação de uma função `main`, responsável por ler um arquivo texto de comandos e escrever um arquivo de saída resultante da execução dos comandos, conforme o funcionamento especificado nas Seções 5.1 e 5.2. Também foram fornecidos arquivos de entrada e as correspondentes saídas esperadas para auxiliar nos testes e verificação se a sua implementação está obedecendo o comportamento esperado. Contudo, a função `main` **não será** considerada na correção e pode ser modificada (ou não) da forma que achar melhor.

2.2 Entrega

A entrega deve ser feita exclusivamente através do eDisciplinas até o dia 4 de outubro.

Você deve entregar exatamente **um** arquivo, chamado `ep_XXXX.c` (em que `XXXX` é o seu número USP). Sugere-se fortemente utilizar o template fornecido como base da implementação.

Novamente, **não é permitido modularizar a solução em mais de um arquivo**. Toda a implementação deve estar contida em `ep_XXXX.c`.

3 Estruturas de Dados

O programa deve utilizar cinco tipos de structs, que devem ser utilizadas conforme a descrição abaixo.

3.1 `celula_t`

Representa um elemento não nulo da planilha. Cada célula participa simultaneamente de duas listas encadeadas: a da sua linha e a da sua coluna. Os seus campos são:

- `linha` e `coluna` – dois inteiros que representam a posição da célula na planilha;

- **valor** – conteúdo inteiro da célula (por definição, sempre diferente de 0);
- **proxima_linha** – ponteiro para a próxima célula na mesma linha, em ordem crescente de coluna;
- **proxima_coluna** – ponteiro para a próxima célula na mesma coluna, em ordem crescente de linha.

Para melhor entendimento desses ponteiros confira a Figura 1.

3.2 fileira_t

Representa a fileira (uma linha ou uma coluna) não vazia, servindo de ponto de entrada para a lista encadeada de células não nulas daquela linha ou coluna. As próprias fileiras formam uma lista ligada ordenada por índice, criada e destruída sob demanda. Desta forma, o maior índice possível das linhas ou colunas da planilha é limitada apenas pelo valor máximo das variáveis inteiras (**INT_MAX**), e não por um valor arbitrário (como seria no caso de pré-estabelecer um **MAX**). Os seus campos são:

- **indice** – número da linha ou coluna representada por esta fileira;
- **primeiro** – ponteiro para a primeira célula não nula daquela linha/coluna (nunca deve ser **NULL**, pois uma fileira só existe enquanto tiver ao menos uma célula);
- **proximo** – ponteiro para a próxima fileira na lista de fileiras, em ordem crescente de índice (deve ser **NULL** para a última fileira).

3.3 operacao_t

Representa um registro da pilha de histórico. Para as operações de definição e remoção, deve-se empilhar as coordenadas e o valor anterior da célula, porém apenas se for diferente do novo valor. Para a operação de transposição, é salvo no histórico as coordenadas iniciais e o tamanho da matriz, porém apenas se ao menos uma célula foi alterada. A struct deve conter os seguintes campos:

- **transposicao** – um booleano que indica se a operação foi uma transposição ou não;
- **linha** e **coluna** – posição da célula afetada pela operação;
- **valor_anterior** (usado apenas quando **transposicao == false**) – valor que a célula tinha antes da operação (0 se ela não existia);
- **tamanho** (usado apenas quando **transposicao == true**) – tamanho da transposição realizada.

Note que os campos **valor_anterior** e **tamanho** nunca são usados ao mesmo tempo, permitindo o uso de um **union**.

3.4 elo_pilha_t

Representa um elo da pilha dinâmica, possuindo os seguintes campos:

- **op** – a operação (**operacao_t**) registrada na pilha;
- **proximo** – um ponteiro para o próximo **elo_pilha_t** da pilha.

3.5 pilha_t

A pilha de operações deve seguir a mesma organização vista em aula para pilhas dinâmicas:

```
1 typedef struct {  
2     elo_pilha_t* topo;  
3 } pilha_t;
```

A pilha de histórico é um campo da própria `planilha_t` (ver Seção 3.6). Consequentemente, as funções que envolvem a pilha não precisam recebê-la como parâmetro, pois podem acessar a pilha diretamente por meio do ponteiro para `planilha_t`.

3.6 planilha_t

Representa a planilha completa. Em vez de vetores de tamanho fixo, a planilha guarda apenas os ponteiros para o início de cada lista de fileiras, além da própria pilha de histórico. Os seus campos são:

- `primeira_linha` – ponteiro para a primeira linha (`fileira_t`) não vazia (NULL se a planilha estiver vazia);
- `primeira_coluna` – ponteiro para a primeira coluna (`fileira_t`) não vazia (NULL se a planilha estiver vazia);
- `total_celulas` – número de células não nulas presentes;
- `historico` – estrutura do tipo `pilha_t` que registra o histórico de alterações realizadas na planilha, usada para viabilizar o `desfazer`.

3.7 Exemplo de representação da estrutura

A figura a seguir ilustra a estrutura cruzada para uma planilha com as células $(0, 1) = 12$, $(2, 0) = 5$, $(1, 1) = 10$ e $(1, 3) = 7$:

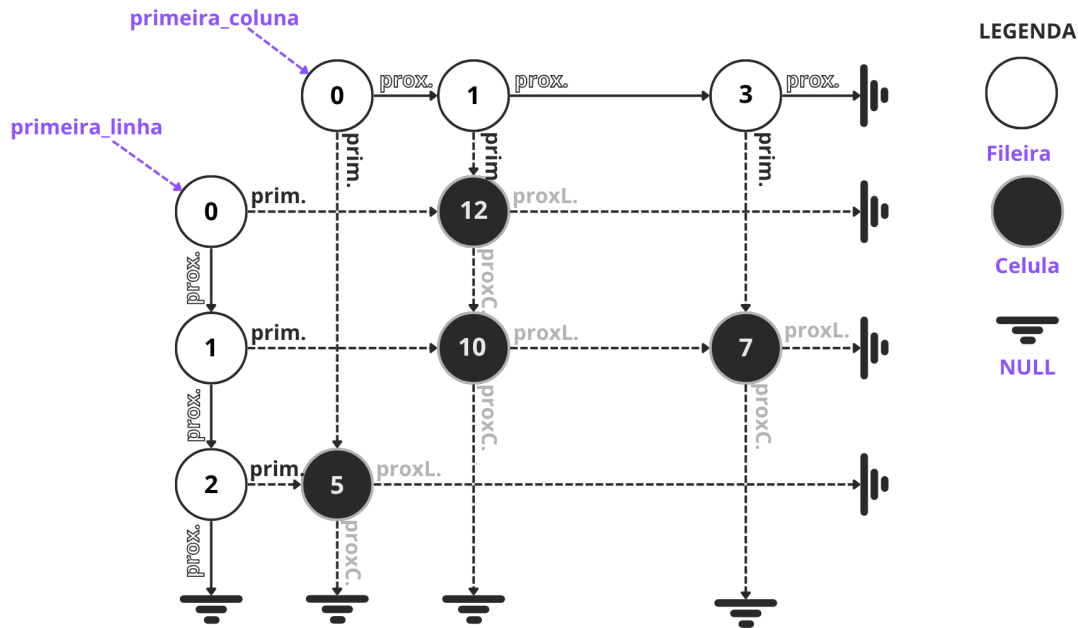


Figura 1: Matriz Esparsa

4 Funções obrigatórias

Esta seção especifica todas as funções que seu programa deve implementar obrigatoriamente, sem modificar a sua assinatura. Outras funções auxiliares podem ser escritas.

Para todas as funções, considere que os índices de linha e coluna estão no intervalo $[0, \text{INT_MAX}]$ (não serão usados valores negativos nos testes).

4.1 void inicializar_planilha(planilha_t* p)

Inicializa a planilha, atribuindo os valores iniciais em todos os campos.

4.2 int obter_valor(planilha_t* p, int linha, int coluna)

Retorna o valor da célula existente na posição (linha, coluna), ou 0 caso ela não exista.

4.3 int somar_intervalo(planilha_t* p, int linha_ini, int linha_fim, int coluna_ini, int coluna_fim)

Retorna a soma dos valores das células cujas linhas e colunas estejam, respectivamente, dentro de $[linha_ini, linha_fim]$ e $[coluna_ini, coluna_fim]$.

4.4 int contar_nao_nulas(planilha_t* p)

Retorna a quantidade de células não nulas armazenadas na planilha.

```

4.5  celula_t* buscar_celula(planilha_t* p, int lin, int col,
    celula_t** cel_ant_linha, celula_t** cel_ant_coluna,
    fileira_t** fil_ant_linha, fileira_t** fil_ant_coluna)

```

Localiza a célula (*lin*, *col*) e retorna um ponteiro para ela, ou NULL caso ela não exista. Além do retorno, a função devolve por referência quatro antecessores, usados tanto para localizar a célula quanto para saber onde inserir uma célula ou uma fileira nova, caso ainda não existam:

- ***cel_ant_linha** – antecessor da célula na lista da **linha** *lin*. Recebe NULL se a célula for (ou, caso não exista ainda, se ela *fosse*) a primeira célula dessa linha, ou se a linha *lin* ainda nem tiver uma fileira;
- ***cel_ant_coluna** – antecessor da célula na lista da **coluna** *col*. Mesma lógica de ***cel_ant_linha**, mas para a coluna;
- ***fil_ant_linha** – antecessor da **fileira de linha** *lin* na lista de fileiras de linha. Recebe NULL se essa fileira for a primeira da lista, ou se a linha *lin* ainda não tiver fileira alguma;
- ***fil_ant_coluna** – antecessor da **fileira de coluna** *col* na lista de fileiras de coluna. Mesma lógica de ***fil_ant_linha**, mas para a coluna.

Um exemplo visual destes valores é exibido na Figura 2, ilustrando a busca pela célula na posição (1,3) em uma planilha com as células $(0,1) = 12$, $(2,0) = 5$, $(1,1) = 10$ e $(1,3) = 7$.

Esses quatro ponteiros resolvem, juntos, dois problemas comuns:

- **Onde inserir a célula** nas listas de linha e coluna (**cel_ant_linha** e **cel_ant_coluna** apontam exatamente para o nó depois do qual a nova célula deve entrar);
- **Se é preciso criar uma fileira nova** para a linha e/ou a coluna, e onde inserir essa fileira na lista de fileiras (**fil_ant_linha** e **fil_ant_coluna** resolvem isso sem precisar de uma segunda busca).

4.7 `bool remover_celula(planilha_t* p, int lin, int col)`

Remove a célula (lin, col) , caso ela exista, registrando a operação no histórico da mesma forma que `definir_celula`.

4.8 `bool transpor(planilha_t* p, int lin, int col, int tamanho)`

Transpõe uma matriz quadrada que está localizada entre as linhas $[lin, lin + tamanho)$ e colunas $[col, col + tamanho)$. Esta função adiciona apenas um item no histórico e retorna `true` se ao menos um valor for alterado durante a transposição. O item do histórico deve possuir `transposicao = true`, além dos valores de linha, coluna e tamanho.

A função também não faz nada e retorna `false` se `lin + tamanho` ou `col + tamanho` resultar em overflow de valor inteiro.

Por exemplo, considere a planilha `p` abaixo:

```
1 2 3 4
5 6 7 8
9 0 0 1
2 3 4 5
```

Após executar `transpor(p, 1, 2, 2)`, o resultado esperado é:

```
1 2 3 4
5 6 7 0
9 0 8 1
2 3 4 5
```

4.9 `bool desfazer(planilha_t* p)`

Desfaz a última operação registrada em `p->historico`, restaurando o estado anterior da célula afetada ou desfazendo a transposição, e remove esse registro do topo da pilha. Retorna `false` se o histórico estiver vazio (nada a desfazer) e `true` caso contrário.

4.10 `void exibir_planilha(planilha_t* p)`

Exibe todas as células não nulas da planilha, uma por linha, no formato `linha coluna valor`, varrendo as linhas em ordem crescente e, dentro de cada linha, as colunas em ordem crescente.

4.11 `void exibir_historico(planilha_t* p)`

Exibe o histórico de operações armazenado em `p->historico`, do topo para a base, imprimindo uma operação por linha. As operações de definição de célula devem ser exibidas no formato `linha coluna valor_anterior`, enquanto que as operações de transposição devem ser exibidas no formato `T linha coluna tamanho` (a letra T indica operação de transposição).

4.12 void liberar_tudo(planilha_t* p)

Libera toda a memória utilizada pela planilha (células e fileiras) e pelo histórico (todos os nós de `p->historico`). Além disso, reinicializa os campos da estrutura para representar uma planilha vazia.

5 Especificação dos arquivos de entrada e saída

O template fornecido possui a funcionalidade de processar um arquivo texto de entrada e produzir um arquivo texto de saída. Esta funcionalidade tem o objetivo de facilitar o teste e a comparação de resultados entre os alunos. Desta forma, você pode compartilhar os arquivos de entrada com seus amigos e comparar os arquivos de saída para ver se produzem o mesmo resultado. Além disso, foram fornecidos alguns exemplos de arquivos de entrada e as respectivas saídas.

A execução do programa segue o seguinte formato:

```
./ep_XXXX arquivo_entrada.txt arquivo_saida.txt
```

Cuidado! Se o arquivo de saída já existir, o seu conteúdo será perdido.

5.1 Arquivo de entrada

O arquivo de entrada possui um comando por linha, no seguinte formato:

- `DEF lin col valor` – equivalente a chamar `definir_celula` com os parâmetros informados;
- `REM lin col` – equivalente a chamar `remover_celula`;
- `GET lin col` – consulta o valor da célula (`obter_valor`);
- `SOMA li lf ci cf` – soma o intervalo $[li, lf] \times [ci, cf]$ (`somar_intervalo`);
- `CONT` – consulta o número de células não nulas (`contar_nao_nulas`);
- `TRANS lin col tam` – transpõe a matriz quadrada com canto superior esquerdo (`lin`, `col`) e dimensão `tam`;
- `DESFAZER` – desfaz a última operação (`desfazer`);
- `EXIBIR` – exibe o estado atual da planilha (`exibir_planilha`);
- `HIST` – exibe o histórico de alterações (`exibir_historico`).

Os comandos são processados na ordem em que aparecem no arquivo de entrada.

Exemplo de entrada:

```
DEF 0 1 12
DEF 0 3 5
DEF 2 1 7
GET 0 1
SOMA 0 2 0 3
```

```
CONT
REM 0 3
DESFAZER
EXIBIR
HIST
```

5.2 Arquivo de saída

Os comandos especificados na seção anterior produzem as seguintes saídas ao serem processados:

- `GET lin col:` imprime uma linha com `GET linha coluna valor` (usa `obter_valor`);
- `SOMA li lf ci cf:` imprime uma linha com `SOMA li lf ci cf` e o valor retornado por `somar_intervalo`;
- `CONT:` imprime uma linha com `CONT` e o valor retornado por `contar_ao_nulas`;
- `DESFAZER:` se a pilha de histórico estiver vazia, imprime `"HISTORICO VAZIO"`; caso contrário, não produz saída;
- `EXIBIR:` imprime `PLANILHA` seguido de uma linha `linha coluna valor` para cada célula não nula, na ordem descrita na Seção 4; se a planilha estiver vazia, imprime apenas `"PLANILHA VAZIA"`;
- `HIST:` imprime `HISTORICO` seguido de uma linha para cada operação na pilha no formato definido na Seção 4.11, do topo para a base; se o histórico estiver vazio, imprime apenas `"HISTORICO VAZIO"`.
- Os comandos `DEF`, `REM` e `TRANS` não geram saída.

A saída padrão do programa é redirecionada para o arquivo de saída. Então, o uso de `printf` irá interferir no arquivo de saída.

Exemplo de saída referente ao exemplo de entrada da Seção 5.1:

```
GET 0 1 12
SOMA 0 2 0 3 24
CONT 3
PLANILHA
0 1 12
0 3 5
2 1 7
HISTORICO
2 1 0
0 3 0
0 1 0
```

6 Correção e atribuição de nota

Serão realizados testes para verificar o correto funcionamento das funções especificadas na Seção 4. A função `main` do seu programa será ignorada e não influenciará na nota (porém não deve apresentar erros de compilação!). A nota do EP será proporcional à quantidade de testes corretos.

Durante a correção, seu programa será compilado em ambiente Linux através do seguinte comando:

```
gcc -Wall -o ep_XXXX ep_XXXX.c
```

Erros de compilação impedem a correção do programa, resultado em nota zero.

6.1 Competição e Premiação

Os programas que receberem nota 10 na correção serão submetidos a um benchmark, que irá realizar diversas operações em uma planilha. Os programas serão classificados de acordo com o tempo de execução do benchmark, e o autor do programa com menor tempo de execução será o vencedor da competição e coroado GRANDE MESTRE DAS ESTRUTURAS LINEARES 2026! Além do título, o vencedor ganhará um prêmio simbólico.

7 Observações

- Em caso de dúvidas, utilize o fórum do edisciplinas (porém não poste trechos de código) ou contate um dos monitores;
- O código deve ser implementado na linguagem C;
- As listas de células (por linha e por coluna) e a lista de fileiras devem ser mantidas sempre ordenadas por índice crescente;
- Note que o nome do arquivo `ep_XXXX.c` deve ter, no lugar do XXXX, o seu número USP;
- Os arquivos devem ser postados individualmente no e-disciplinas (não compactá-los!), sem pastas ou subpastas;
- Os EPs que apresentarem problemas de compilação, incluindo por não seguir o formato das estruturas e assinatura das funções, terão sua pontuação zerada;
- Qualquer evidência de plágio entre trabalhos, mesmo que parcial, implicará na nota zero no trabalho para todos os envolvidos;
- Qualquer evidência de resolução do EP com a utilização de IA, resultará na anulação do trabalho por completo.

Espero que se divirtam ;D